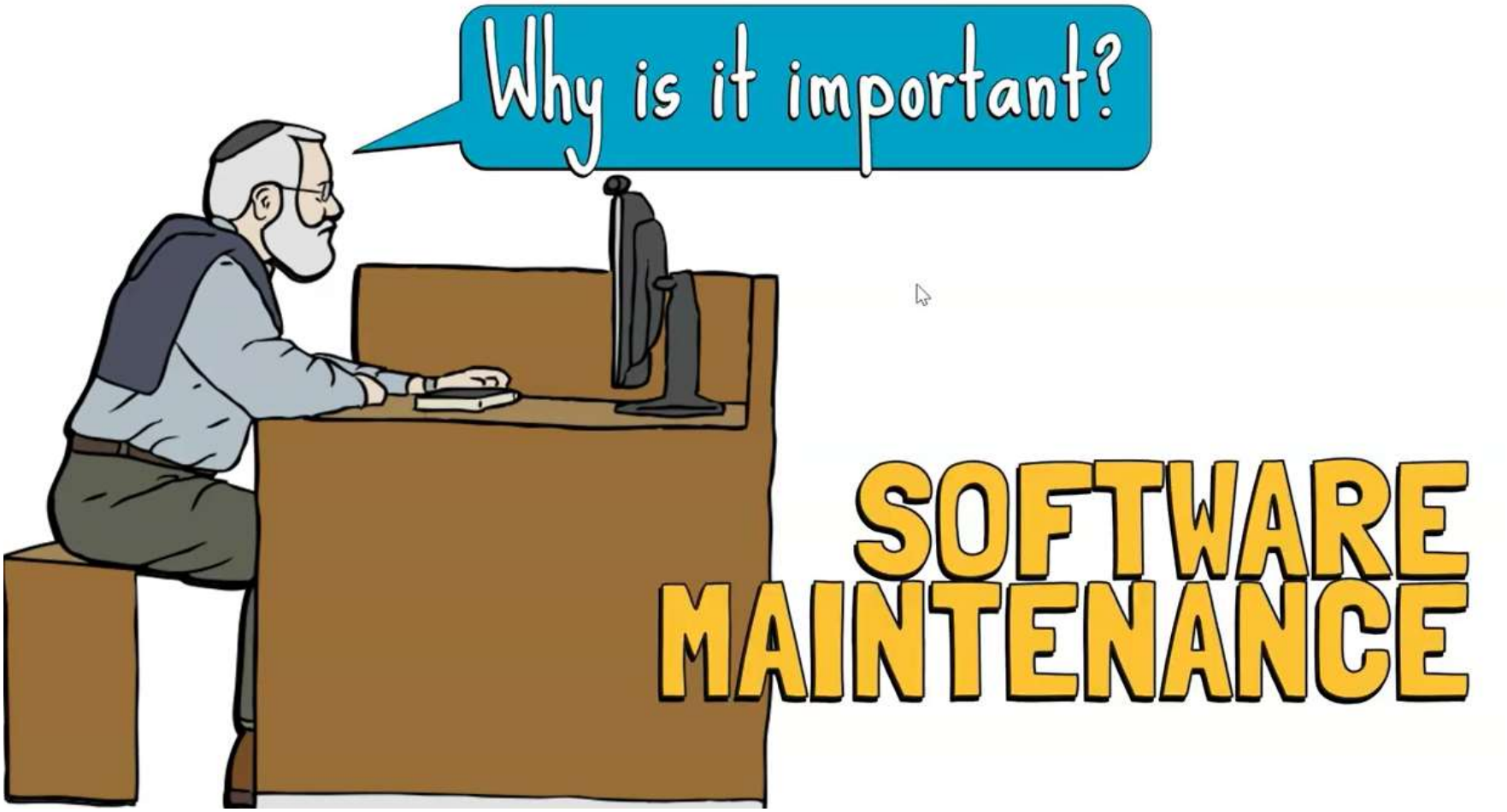
Unit 6: Implementation and Maintenance



By: Kabita Dhital
BIM 5th Sem

System Implementation

- Implementation is a process of ensuring that the information system is operational. It involves:
- Constructing a new system from scratch
- Constructing a new system from the existing one.
- Implementation allows the users to take over its operation for use and evaluation.
- It involves training the users to handle the system and plan for a smooth conversion.
- Implementation is a process of ensuring that the information system is operational.
- It involves:

- Constructing a new system from scratch
- Constructing a new system from the existing one.
- Implementation allows the users to take over its operation for use and evaluation.
- It involves training the users to handle the system and plan for a smooth conversion.

System Implementation

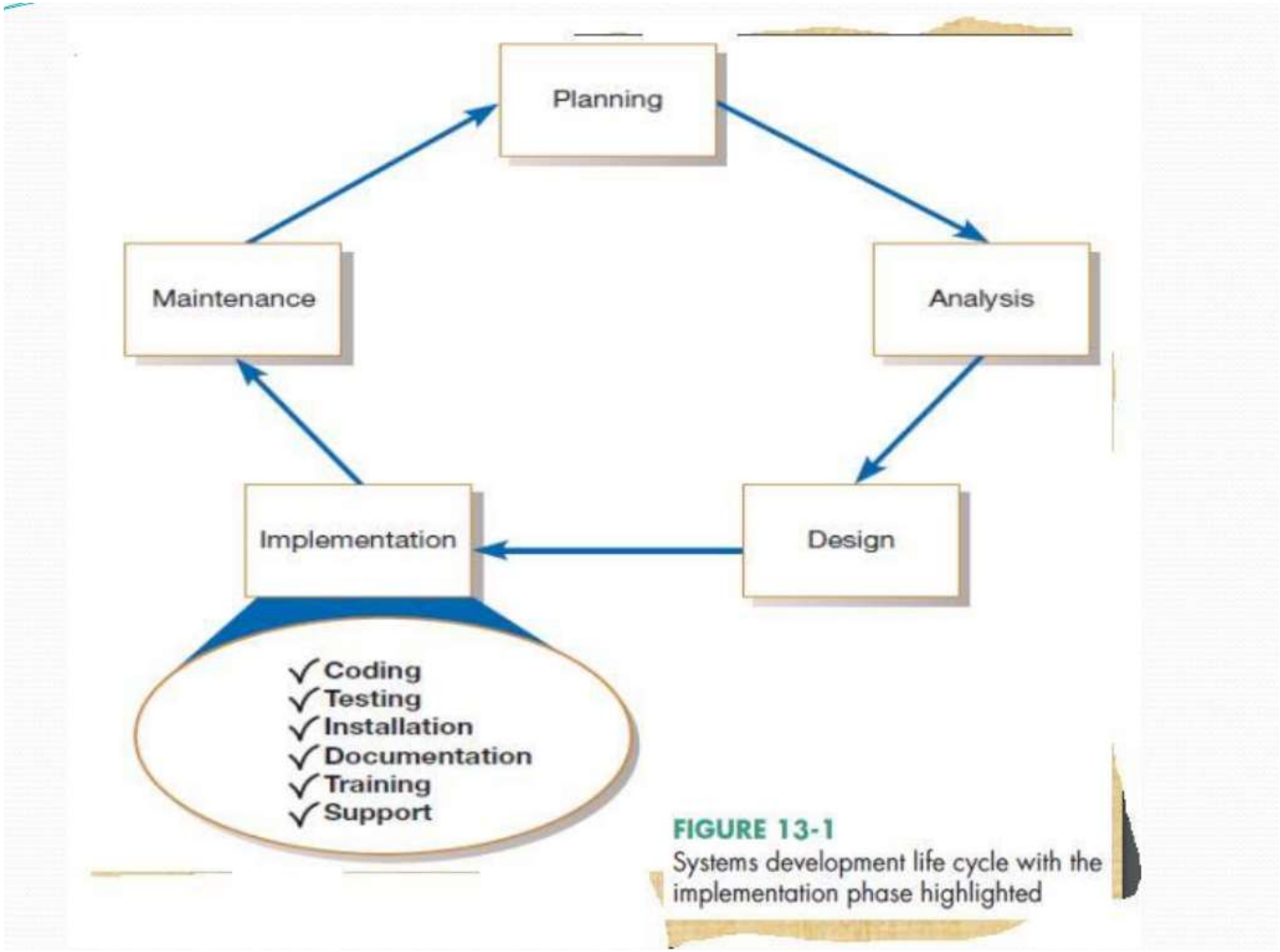
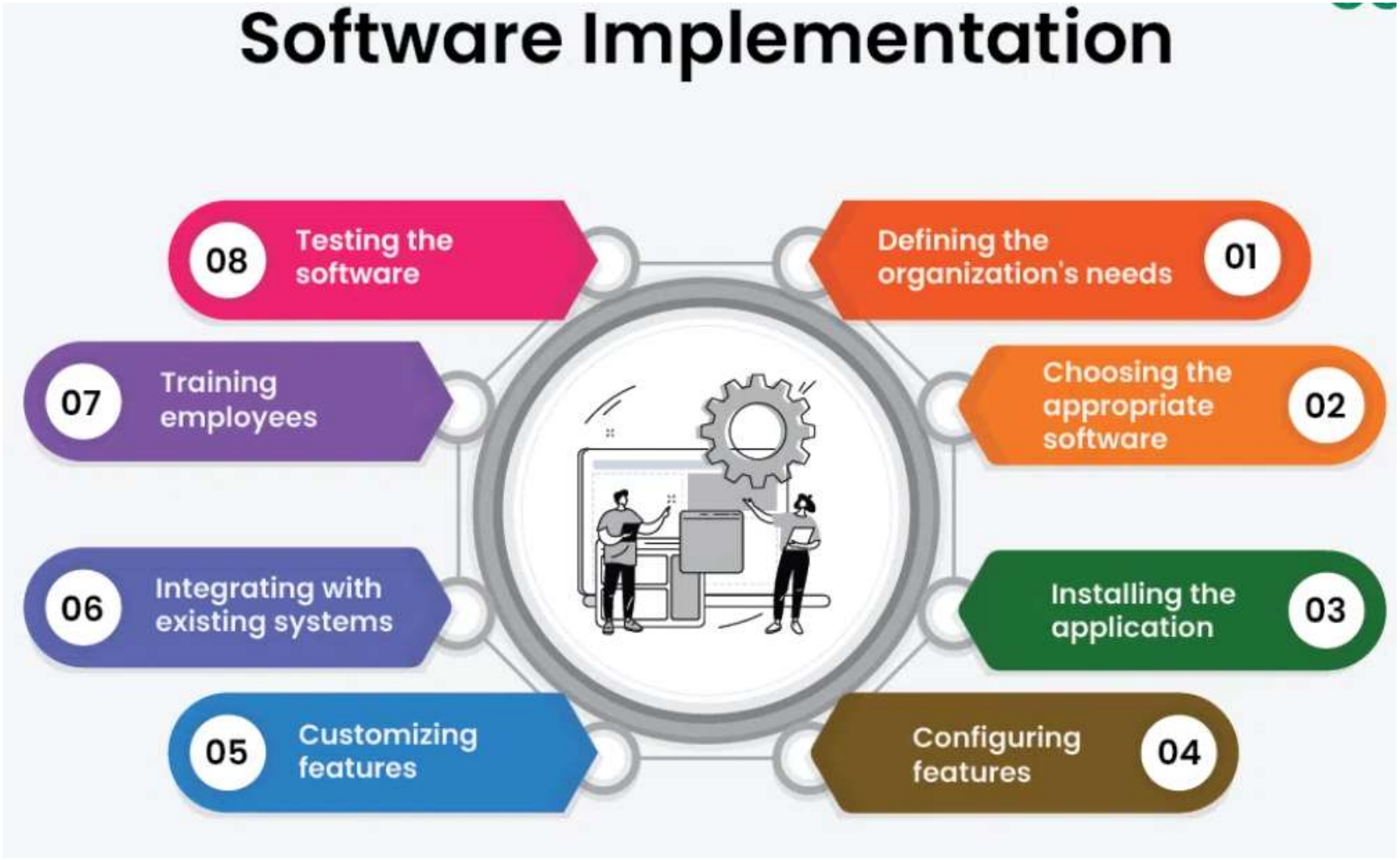


FIGURE 13-1
Systems development life cycle with the implementation phase highlighted

Software Implementation



- System implementation is made up of many activities. The six major activities we are concerned with this are coding, testing, installation, documentation, training, and support (see Figure 13-1). The purpose of these steps is to convert the physical system specifications into working and reliable software and hardware, document the work that has been done, and provide help for current and future users and caretakers of the system.
- Coding and testing may have already been completed by this point if Agile Methodologies have been followed. Using a plan-driven methodology, coding and testing are often done by other project team members besides analysts, although analysts may do some programming. In any case, analysts are responsible for ensuring that all of these various activities are properly planned and executed. Next, we will briefly discuss these activities in two groups:

1. Coding, testing, and installation and
2. Documenting the system and training and supporting users.

Coding, testing, and Installation processes

- Coding is the process whereby the physical design specifications created by the analysis team are turned into working computer code by the programming team. Depending on the size and complexity of the system, coding can be an involved, intensive activity. once coding has begun, the testing process can begin and proceed in parallel. As each program module is produced, it can be tested individually, then as part of a larger program, and then as part of a larger system.
- Installation is the process during which the current system is replaced by the new system. This includes conversion of existing data, software, documentation, and work procedures to those consistent with the new system. Users will sometimes resist these changes, and you must help them adjust. However, you cannot control all the dynamics of user–system interaction involved in the installation process.

- The Processes of Documenting the System, Training Users, and Supporting Users: Although the process of documentation proceeds throughout the life cycle, it receives formal attention during the implementation phase because the end of implementation largely marks the end of the analysis team's involvement in systems development. As the team is getting ready to move on to new projects, you and the other analysts need to prepare documents that reveal (give information) all of the important information you have accumulated about this system during its development and implementation. There are two audiences for this final documentation:
- The information systems personnel who will maintain the system throughout its productive life, and
- The people who will use the system as part of their daily lives. The analysis team in a large organization can get help in preparing documentation from specialized staff in the information systems department.
- Larger organizations also tend to provide training and support to computer users throughout the organization. Some of the training and support is very specific to particular application systems, whereas the rest is general to particular operating systems or off-the-shelf software packages.

Software application testing

- Software testing can be stated as the process of verifying and validating that a software or application is bug free, meets the technical requirements as guided by its design and development and meets the user requirements effectively and efficiently with handling all the exceptional and boundary cases. Software testing is method of assessing the functionality of a software program. The process of software testing aims not only at finding faults in the existing software but also at finding measures to improve the software in terms of efficiency, accuracy and usability. It mainly aims at measuring specification, functionality and performance of a software program or application.
- **Static testing means** that the code being tested is not executed. The results of running the code are not an issue for that particular test. **Dynamic testing**, on the other hand, involves execution of the code. **Automated testing** means the computer conducts the test, whereas **manual testing** means that people complete the test

1. Static Testing

- **Definition:** Testing **without executing the code**.
- **Goal:** Find errors in code, design, or documentation early.
- **Examples:**
  - Code reviews
  - Walkthroughs
  - Inspections
- **Pros:** Detects defects early, cheaper to fix.
- **Cons:** Cannot catch runtime errors.

2. Dynamic Testing

- **Definition:** Testing **by executing the code** to check functionality.
- **Goal:** Validate that the software works as expected.
- **Examples:**
  - Unit Testing
  - Integration Testing
  - System Testing
- **Pros:** Detects runtime errors and functional issues.
- **Cons:** Requires executable software.

3. Manual Testing

- **Definition:** Human testers execute test cases **without automation tools**.
- **Goal:** Simulate real user behavior to find defects.
- **Examples:**
  - Exploratory Testing
  - Ad-hoc Testing
  - User Acceptance Testing (UAT)
- **Pros:** Flexible, good for usability testing.
- **Cons:** Time-consuming, prone to human error.

4. Automated Testing

- **Definition:** Using **software tools/scripts** to execute test cases automatically.
- **Goal:** Reduce repetitive effort and improve accuracy.
- **Examples:**
  - Selenium (web automation)
  - JUnit/TestNG (unit tests)
  - Cypress (end-to-end testing)
- **Pros:** Fast, repeatable, good for regression testing.
- **Cons:** Initial setup is costly; not ideal for usability testing.

Seven Different types of tests

1. Inspections
2. Desk checking
3. Unit testing
4. Integration testing
5. System Testing
6. Stub testing
7. User acceptance testing

1. Inspections

Participants manually examine code for occurrences of well-known errors.

Syntax, grammar, and some other routine errors can be checked by automated inspection software.

So manual inspection checks are used for more subtle (small) errors.

Each programming language lends itself to certain types of errors that programmers make when coding, and these common errors are well known and documented.

It has been estimated that code inspections detect from 60 to 90 percent of all software defects as well as provide programmers with feedback that enables them to avoid making the same types of errors in future work

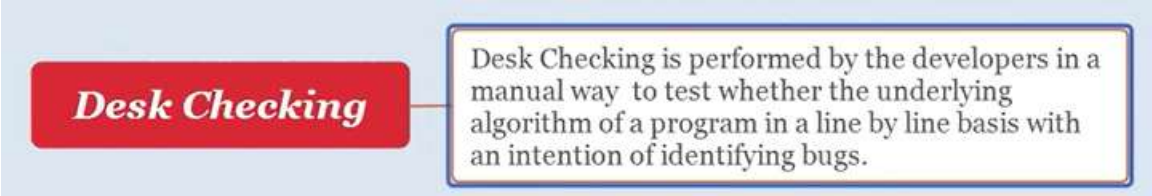
Desk checking

A testing technique in which the program code is sequentially executed manually by the reviewer.

It is informal process in which the programmer or someone else who understands the logic of the program works through the code with a paper and pencil.

The programmer executes each instruction, using test cases that may or may not be written down.

In one sense, the reviewer acts as the computer, mentally checking each step and its results for the entire set of computer instructions.



3. Unit testing

sometimes called module testing,

An automated technique where each module is tested.

But because modules coexist and work with other modules in programs and the system, they must also be tested together in larger groups.

Unit testing benefits software development projects in many ways.

Efficient bug discovery

If there are any input, output, or logic-based errors within a code block, your unit tests help you catch them before the bugs reach production. When code changes, you run the same set of unit tests—alongside other tests such as integration tests—and expect the same results. If tests fail (also called *broken tests*) it indicates regression-based bugs.

Unit testing also helps finds bugs faster in code. Your developers don't spend a large amount of time on debugging activities. They can quickly pinpoint the exact part of the code that has an error.

Documentation

It's important to document code to know exactly what that code is supposed to be doing. That said, unit tests also act as a form of documentation.

Other developers read the tests to see what behaviors the code is expected to exhibit when it runs. They use the information to modify or refactor the code. Refactoring code makes it more performant and well-composed. You can run the unit testing again to check that code works as expected after changes.

4. Integration testing

Combining modules and testing them is called integration testing.

Integration testing is gradual.

First you test the coordinating module and only one of its subordinate modules.

After the first test, you add one or two other subordinate modules from the same level.

Once the program has been tested with the coordinating module and all of its immediately subordinate modules, you add modules from the next level and then test the program.

You continue this procedure until the entire program has been tested as a unit.

5.System Testing

- System Testing is done to check whether the software or product meets the specified requirements or not.
- It is done by both testers and developers.
- It contains the Testings: System testing, Integration Testing.

5.1 White-box testing

Testing technique in which software's internal structure, design, and coding are tested to verify input-output flow and improve design, usability, and security.

Code is visible to testers, so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing, and Glass box testing.

White Box Testing

- White Box Testing,or glass box testing, is a software testing technique that focuses on the software's internal logic, structure, and coding. It provides testers with complete application knowledge, including access to source code and design documents, enabling them to inspect and verify the software's inner workings, infrastructure, and integrations.
- Test cases are designed using an internal system perspective, and the methodology assumes explicit knowledge of the software's internal structure and implementation details. This in-depth visibility allows White Box Testing to identify issues that may be invisible to other testing methods.
- Unlike black box testing, which focuses on testing the software's functionality without knowledge of its internal workings, white box testing involves looking inside the application and understanding its code, logic, and structure.

Tools and Frameworks for White Box Testing

- JUnit
- NUnit
- TestNG
- BrowserStack Code Quality
- pytest
- xUnit.net
- Eclipse
- IntelliJ IDEA
- Visual Studio

5.2 Black-box Testing

- Black Box Testing is testing technique having no knowledge of the internal functionality/structure of the system
- It is also called Behavioural, Functional, Opaque-Box, Closed-Box etc
- Focuses on testing the function of the program or application against its specifications
- Determines whether combinations of inputs and operations produce expected results

Black box testing

- Black box testing involves testing a system with no prior knowledge of its internal workings. A tester provides an input, and observes the output generated by the system under test. This makes it possible to identify how the system responds to expected and unexpected user actions, its response time, usability issues and reliability issues.
- Black box testing is a software testing method that evaluates functionality based on requirements and specifications without knowledge of the internal code or structure. Focused on user experience, it verifies inputs and outputs to ensure the software behaves as expected.

6. Stub testing

Stubs are two or three lines of code written by a programmer to stand in for the missing modules.

During testing, the coordinating module calls the stub instead of the subordinate module.

The stub accepts control and then returns it to the coordinating module.

7. User Acceptance Testing

- Testing is done after the system testing.
- Checks whether the software meets the customer's requirements or not.
- Acceptance testing is used by testers, stakeholders as well as clients.
- It includes only Functional Testing and it contain two testing Alpha Testing and Beta Testing.

Installation

- The process of moving from the current information system to the new one is called installation.
- All employees who use a system, whether they were consulted during the development process or not, must give up their reliance on the current system and begin to rely on the new system.
- Four different approaches to installation have emerged over the years: direct, parallel, single-location, and phased .
- The approach an organization decides to use will depend on the scope and complexity of the change associated with the new system and the organization's risk aversion.

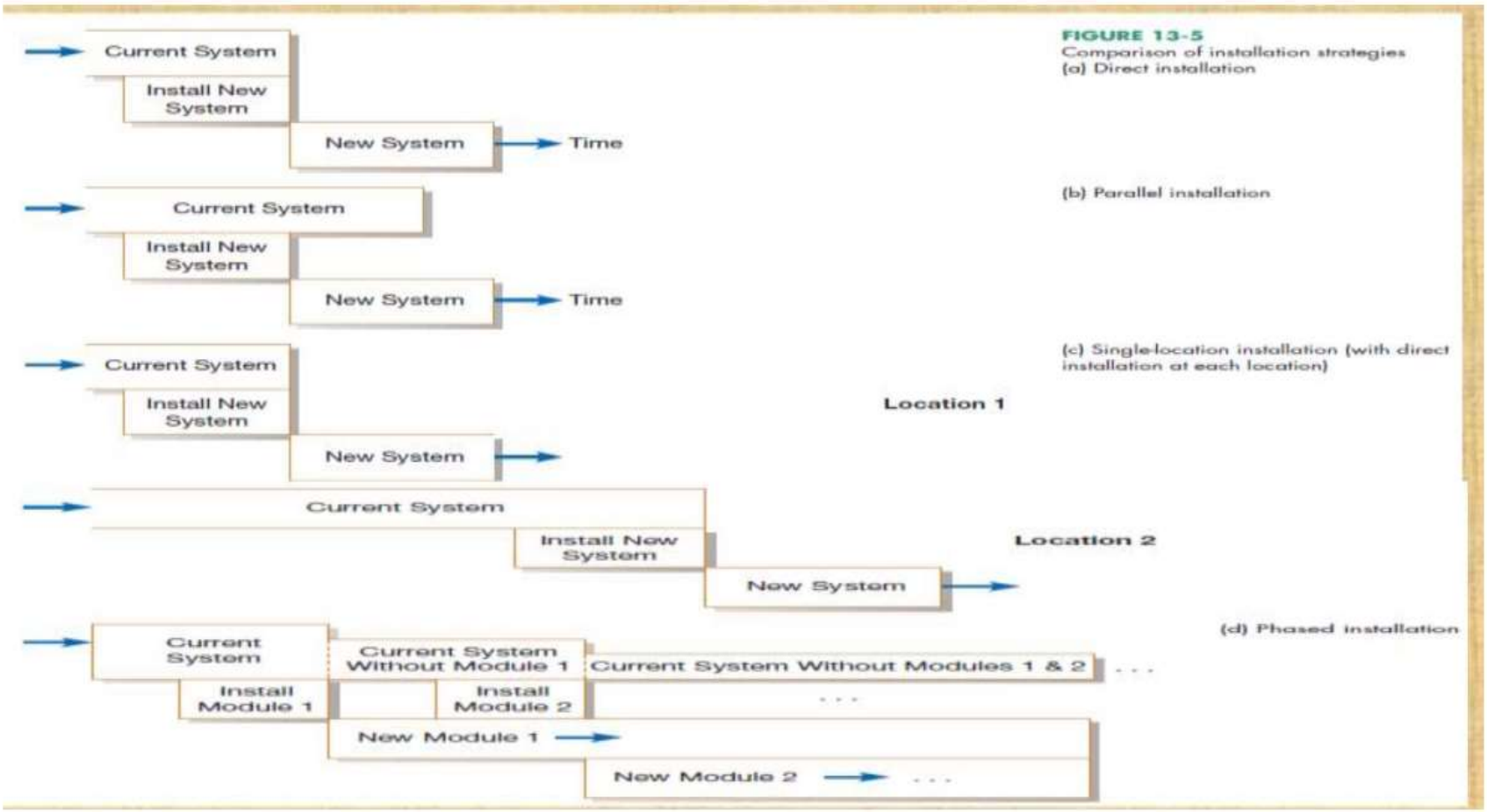


FIGURE 13-5
Comparison of installation strategies
(a) Direct installation

(b) Parallel installation

(c) Single-location installation (with direct installation at each location)

(d) Phased installation

Direct Installation

- The direct, or abrupt, approach to installation (also called “cold turkey”) is as sudden as the name indicates: the old system is turned off and the new system is turned on.
- Old system is **completely replaced** by the new system at once .
- Under direct installation, users are at the mercy of the new system.
- Any errors resulting from the new system will have a direct impact on the users and how they do their jobs and, in some cases—depending on the centrality of the system to the organization—on how the organization performs its business.
- If the new system fails, considerable delay may occur until the old system can again be made operational and business transactions are reentered to make the database up to date.
- For these reasons, direct installation can be very risky.
- Further, direct installation requires a complete installation of the whole system.
- For a large system, this may mean a long time until the new system can be installed, thus delaying system benefits or even missing the opportunities that motivated the system request.
- On the other hand, it is the least expensive installation method, and it creates considerable interest in making the installation a success.
- Sometimes, a direct installation is the only possible strategy if there is no way for the current and new systems to coexist, which they must do in some way in each of the other installation approaches.

Parallel installation

- Parallel installation is as riskless as direct installation is risky.
- Under parallel installation, the old system continues to run alongside the new system until users and management are satisfied that the new system is effectively performing its duties and the old system can be turned off.
- All of the work done by the old system is concurrently performed by the new system.
- Outputs are compared to help determine whether the new system is performing as well as the old.
- Errors discovered in the new system do not cost the organization much, if anything, because errors can be isolated and the business can be supported with the old system.
- Because all work is essentially done twice, a parallel installation can be very expensive; running two systems implies employing (and paying) two staffs to not only operate both systems, but also to maintain them.
- A parallel approach can also be confusing to users because they must deal with both systems.
- As with direct installation, there can be a considerable delay until the new system is completely ready for installation.

Parallel installation

- A parallel approach may not be feasible, especially if the users of the system (such as customers) cannot tolerate redundant effort or if the size of the system (number of users or extent of features) is large.

Single-location installation

- Single-location installation, also known as location or pilot installation, is a middle of- the-road approach compared with direct and parallel installation.
- Rather than convert all of the organization at once, single location installation involves changing from the current to the new system in only one place or in a series of separate sites over time.
- The single location may be a branch office, a single factory, or one department, and the actual approach used for installation in that location may be any of the other approaches.
- The key advantage to single-location installation is that it limits potential damage and potential cost by limiting the effects to a single site.
- Once management has determined that installation has been successful at one location, the new system may be deployed in the rest of the organization, possibly continuing with installation at one location at a time.
- Success at the pilot site can be used to convince reluctant personnel at other sites that the system can be worthwhile for them as well.
- Problems with the system (the actual software as well as documentation, training, and support) can be resolved before deployment to other sites.

Single-location installation

- Even though the single-location approach may be simpler for users, it still places a large burden on information systems (IS) staff to support two versions of the system.
- On the other hand, because problems are isolated at one site at a time, IS staff members can devote all of their efforts to success at the pilot site.
- Also, if different locations require sharing of data, extra programs will need to be written to synchronize the current and new systems; although this will happen transparently to users, it is extra work for IS staff.
- As with each of the other approaches (except phased installation), the whole system is installed; however, some parts of the organization will not get the benefits of the new system until the pilot installation has been completely tested.

Phased installation

- Phased installation, also called staged installation, is an incremental approach. With phased installation, the new system is brought online in functional components; different parts of the old and new systems are used in cooperation until the whole new system is installed. (Figure 13-5d shows the phase-in of the first two modules of a new system.) Phased installation, like single-location installation, is an attempt to limit the organization's exposure to risk, whether in terms of cost or disruption of the business.
- By converting gradually, the organization's risk is spread out over time and place. Also, a phased installation allows for some benefits from the new system before the whole system is ready. For example, new data-capture methods can be used before all reporting modules are ready.
- For a phased installation, the new and replaced systems must be able to coexist and probably share data.
- Thus, bridge programs connecting old and new databases and programs often must be built. Sometimes, the new and old systems are so incompatible (built using totally different structures) that pieces of the old system cannot be incrementally replaced, so this strategy is not feasible.
- On the other hand, each phase of change is smaller and more manageable for all involved

Documenting the System

- System documentation is the comprehensive process of recording a system's requirements, design, architecture, and code, crucial for maintenance, troubleshooting, and onboarding. It ensures smooth operation by bridging communication between developers and users, covering everything from technical specifications to user manuals.
- System cannot be considered to be complete, until it is properly documented.
- Proper documentation of system is necessary due to the following reasons:

1. It solves the problem of indispensability (not willing) of an individual for an organization. Even if the person, who has designed or developed the system, leaves the organization, the documented knowledge remains with the organization, which can be used for the continuity of that software.

2. It makes system easier to modify and maintain in the future. The key to maintenance is proper and dynamic documentation. It is easier to understand the concept of a system from the documented records.

3. It helps in restarting a system development, which was postponed due to some reason. The job need not be started from scratch, and old ideas may still be easily recapitulated from the available documents, which avoids duplication of work, and saves lot of times and effort.

Types of Documentation

1. System Documentation: System documentation records detailed information about a system's design specification, its internal workings and its functionality. System documentation is intended primarily for maintenance programmers.

It contains the following information:

- A description of the system specifying the scope of the problem, the environment in which it functions, its limitation, its input requirements, and form and types of output required.
- Detailed diagram of system flowchart and program flowchart.
- A source listing of all the full details of any modifications made since its development.
- Specification of all input and output media required for the operation of the system.
- Problem definition and the objective of developing the programs.
- Output and test report of the program.
- Upgrade or maintenance history, if modification of the program is made

Two types of system documentation.

i) Internal documentation: Internal documentation is part of the program source code or is generated at compile time.

Written inside the **source code** Helps programmers understand the code logic Includes comments and descriptions.

- **Examples:**
- Comments in code (//, /* */)
- Variable and function descriptions
- Meaningful naming of variables
- Compiler-generated reports

ii) External documentation: External documentation includes the outcome of structured diagramming technique such as dataflow an entity-relationship diagrams.

- **Features:**
- Prepared outside the code
- Helps users, analysts, and developers understand the system
- Often visual or descriptive

Examples:

- Data Flow Diagrams (DFD)
- Entity-Relationship (ER) Diagrams
- System manuals
- User guides

2. User documentation

User documentation consists of **written or visual information** that explains an application system, how it works, and how to use it. It is mainly prepared for **end users**.

User documentation provides instructions and guidance to users on how to operate a system, including setup, troubleshooting, security, and quick reference information.

User documentation typically includes:

- **System setup and operation details**
  → Instructions on how to install, start, and use the system
- **Loading and unloading procedures**
  → Steps for inputting and removing data or files
- **Problems and solutions**
  → Possible errors, their meanings, and corrective actions
- **Security measures and checks**
  → Guidelines for safe and secure system usage
- **Quick reference guides**
  → Short and concise instructions for fast operation

Maintenance

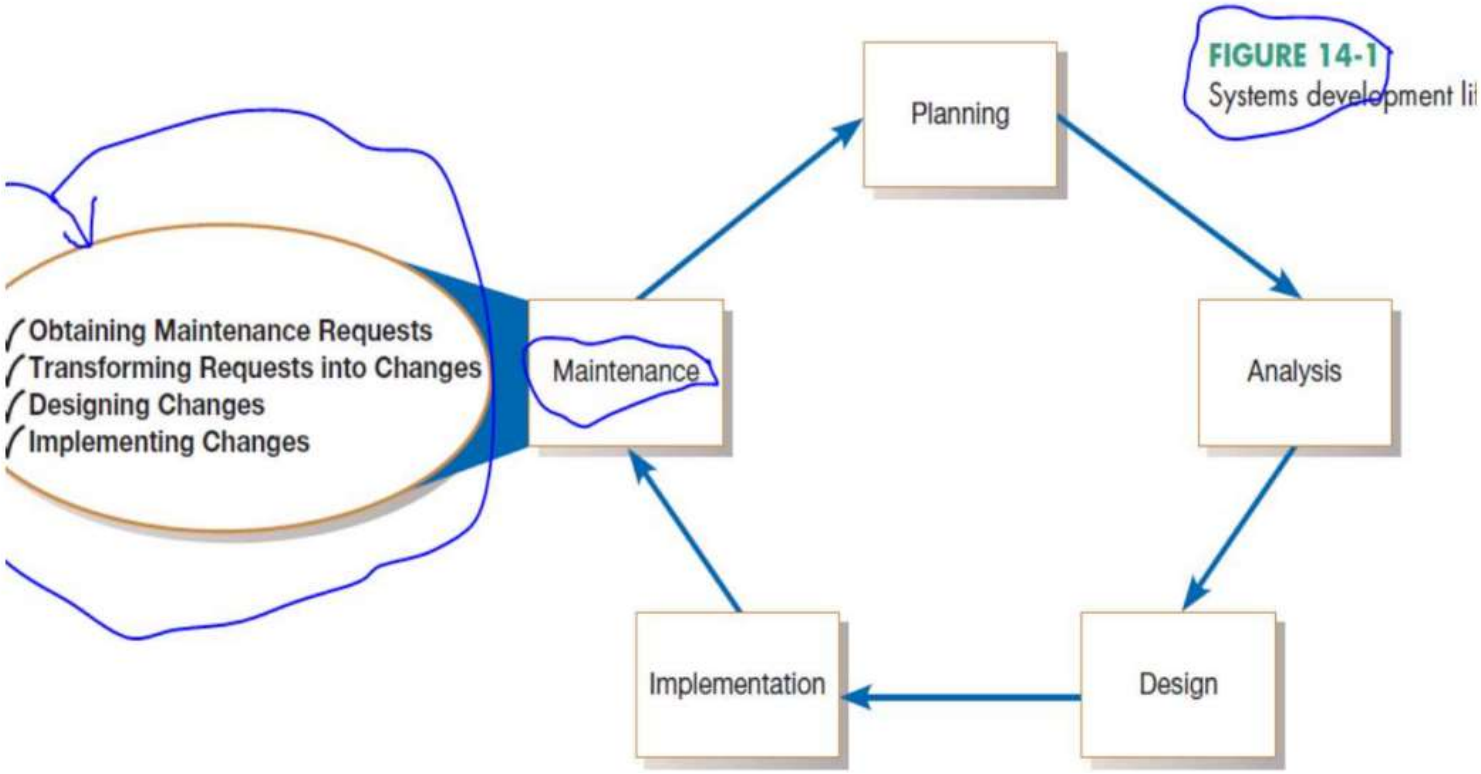


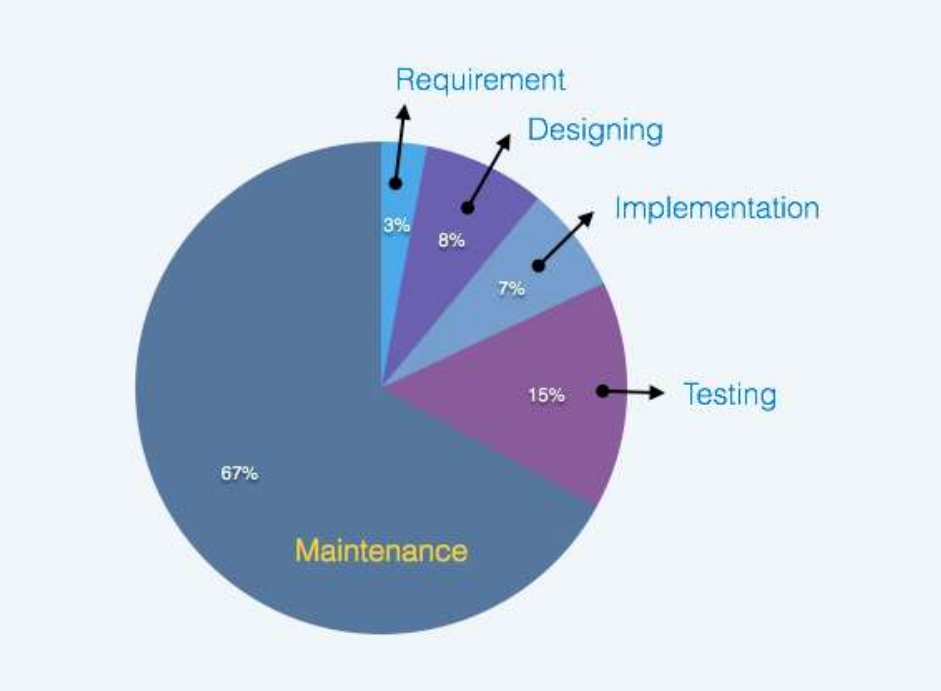
FIGURE 14-1
Systems development li

Four major activities occur within maintenance

1. Obtaining maintenance requests
2. Transforming requests into changes
3. Designing changes
4. Implementing changes

- Once an information system is installed, it enters the **maintenance phase** of the **Systems Development Life Cycle (SDLC)**. In this phase, the system is **monitored, updated, and improved** to meet changing needs.
- **Maintenance Process**
- **Collection of Maintenance Requests**
  - Requests are gathered from:
    - System users
    - System auditors
    - Data center and network staff
    - Data analysts
- **Analysis of Requests**
  - Each request is analyzed to understand:
    - Impact on the system
    - Required changes
    - Business benefits and necessity
- **Approval of Changes**
  - Only necessary and beneficial changes are approved
- **Design and Implementation**
  - Approved changes are designed and implemented in the system
- **Testing and Review**
  - Changes are **formally tested and reviewed** to ensure correctness
- **Installation (Deployment)**
  - After successful testing, changes are installed into the operational system

- The maintenance phase is the last phase of the SDLC.
- It is here that the SDLC becomes a cycle, with the last activity leading back to the first.
- This means that the process of maintaining an information system is the process of returning to the beginning of the SDLC and repeating development steps until the change is implemented.



Obtaining Maintenance Requests

- Obtaining maintenance requests is the process of **collecting suggestions, problems, and change requirements** from users and other stakeholders after a system is in operation.
- Maintenance requests can come from:
- **System users** → report errors or request improvements
- **System auditors** → suggest compliance or control changes
- **IT staff (data center & network team)** → identify technical issues
- **Data analysts** → recommend data-related improvements
- **Management** → request new features or enhancements

Methods of Collecting Requests

- Help desk or support system
- Feedback forms
- Emails or reports
- Issue tracking systems
- Direct user communication

Transforming requests into changes

- After a maintenance request is received, it must be **carefully analyzed** to understand its **scope, impact, and feasibility** before implementation.

Steps in Analysis

- **Understanding the Scope**
  → Determine what the request involves and the extent of changes required
- **Impact Analysis**
  → Identify how the request will affect the **existing system**
- **Time Estimation**
  → Estimate how long the change will take to implement
- **Risk and Feasibility Analysis**
  → Evaluate:
  - Technical feasibility
  - Operational feasibility
  - Possible risks
- **Conversion to Design Change**
  → Approved requests are transformed into a **formal design specification**
- **Forward to Implementation**
  → The design is then sent to the **maintenance implementation phase**

Designing changes

- The activities performed during system maintenance are **very similar to the phases of the Systems Development Life Cycle (SDLC)**.
- Maintenance follows a **mini SDLC cycle**
- The same **concepts and techniques** used in system development are also applied in maintenance
- This similarity ensures consistency and efficiency in managing system changes
- The maintenance process closely resembles the SDLC, where obtaining requests corresponds to planning, analyzing requests to analysis, designing changes to design, and implementing changes to implementation. Thus, maintenance is essentially a continuation of the SDLC.

Explanation

- **Planning Phase**
  → In maintenance, this corresponds to **collecting maintenance requests**
- **Analysis Phase**
  → Requests are analyzed and converted into **specific system changes**
- **Design Phase**
  → Required modifications are **designed**
- **Implementation Phase**
  → Changes are **implemented, tested, and installed**

Implementing changes

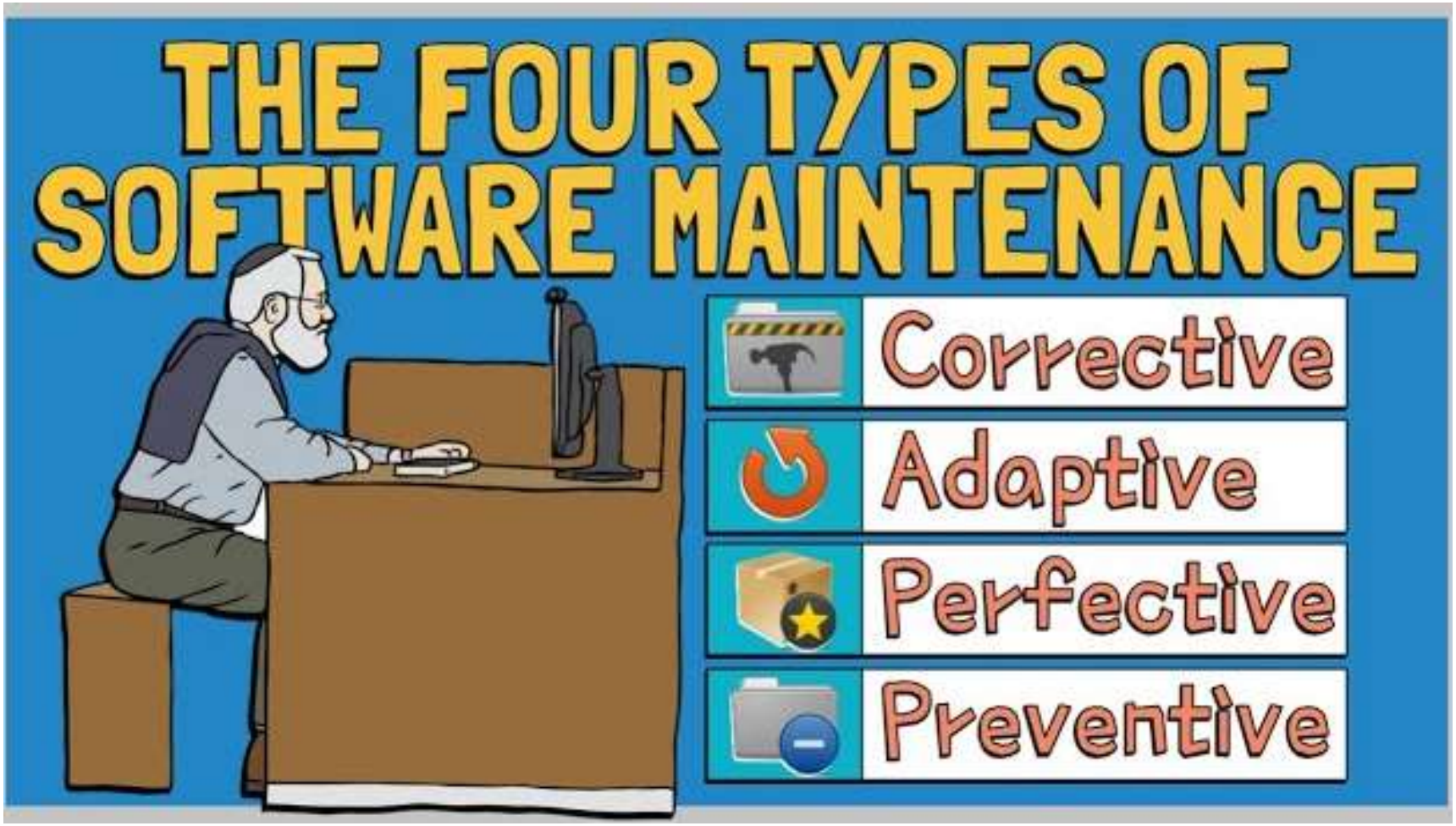
- Implementing changes is the process of **applying approved modifications** to the existing system after analysis and design in the maintenance phase.
- Changes must be **carefully tested before deployment.**
- Helps improve system performance and functionality

Steps in Implementing Changes

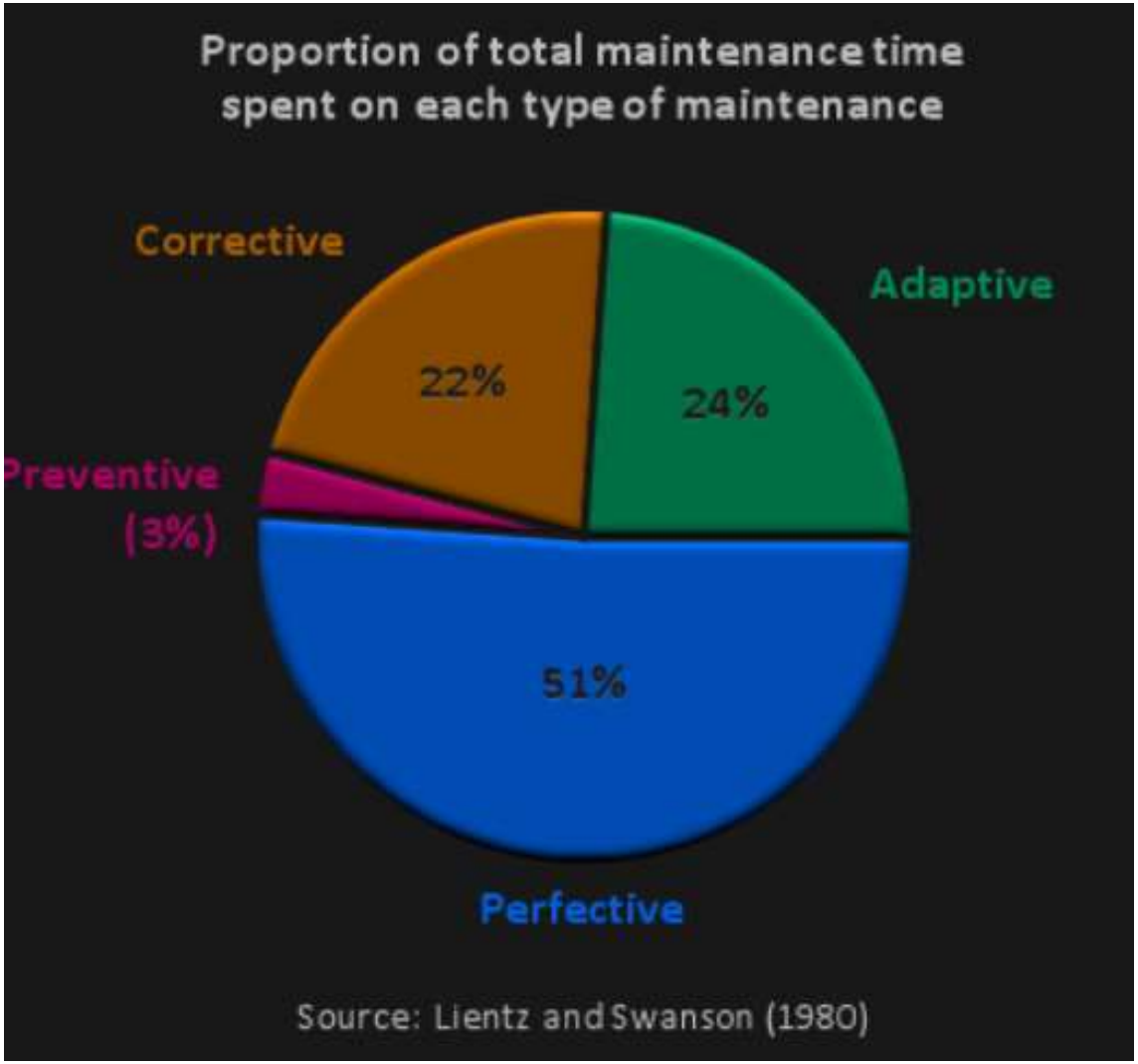
- **Coding the Changes**
  → Modify or add program code based on the design
- **Testing the Changes**
  → Perform testing (unit, integration, system) to ensure correctness
- **Review and Validation**
  → Verify that changes meet requirements and do not create new errors
- **Installation (Deployment)**
  → Install the updated system into the operational environment
- **Documentation Update**
  → Update internal and external documentation to reflect changes

Conducting Systems Maintenance

- A significant within organizations does not go to the development of new systems but to the maintenance of existing systems.



Conducting Systems Maintenance



- **Preventive Maintenance**: Regularly scheduled tasks that prevent equipment failure. This involves taking proactive measures to avoid potential system issues, emphasizing the importance of regular updates and systematic checks to maintain the systems in a state of peak efficiency and effectiveness; such a strategy helps ensure that the systems not only operate smoothly but also remain up-to-date with the latest technological advancements, thereby preventing any future complications.
- **Corrective Maintenance**: Repairs made after a system failure has occurred. This type of maintenance is crucial for corrective software maintenance, focusing on identifying and rectifying existing problems within the system. It typically involves resolving **software glitches** or repairing hardware faults.
- **Adaptive Maintenance**: Modifications made to keep systems updated with new requirements or technologies. This approach focuses on modifying and updating systems to meet evolving requirements. It includes tasks such as updating software for enhanced compatibility. Such maintenance is crucial to ensure the system remains relevant and operates effectively under new conditions.
- **Perfective Maintenance**: Enhancements to improve performance or to modify functionality. Focused on enhancing the system, this type of maintenance involves incorporating new features into software or making strategic improvements. These updates boost system performance and user experience, ensuring the system meets and exceeds user expectations.


Thank you